WanderGuide Server API Reference

Base URL: `https://korea-2026-bot-production.up.railway.app`

Authentication: None (open API) **Content-Type:** `application/json` for all POST/PUT/DELETE requests **CORS:** Enabled (any origin)

---

1. WanderGuide — User Registration & Location

Register a WanderGuide User

Creates or updates a user for proximity alerts.

```
POST /api/wanderguide/register
```

Request Body:

```
{
  "userName": "eran",           // Required. Display name for alerts
  "pushoverKey": "uid2drp..."   // Optional. Pushover user key for native push
}
```

Response:

```
{
  "success": true,
  "userId": 123456789,          // Synthetic user ID (use this for subsequent calls)
  "userName": "eran",
  "pushoverRegistered": true
}
```

Notes:

- If `pushoverKey` is provided, a test notification is sent immediately
- The `userId` returned should be stored and used for all subsequent API calls
- Calling again with the same `userName` updates the existing registration

---

Send User Location

Send GPS coordinates for a user. Triggers proximity check automatically.

```
POST /api/wanderguide/location
```

Request Body:

```
{
  "userId": 123456789,     // Required (from registration response)
  "userName": "eran",      // Optional (for display in alerts)
  "lat": 37.5796,          // Required. Latitude
  "lng": 126.9770          // Required. Longitude
}
```

Response:

```
{
  "success": true,
  "userId": 123456789
}
```

Notes:

- Call this every 15-30 seconds for real-time proximity alerts
- Each call triggers a proximity check against the POI database
- If the user is near a POI and cooldown has passed:
  - Pushover notification sent (if registered)
  - Alert added to the alert queue
  - Alert sent to Telegram DM (if registered via Telegram)

Enable/Disable WanderGuide

Toggle proximity alerts on/off for a user.

```
POST /api/wanderguide/toggle
```

Request Body:

```
{
  "userId": 123456789,
  "userName": "eran",
  "enabled": true           // true = alerts ON, false = alerts OFF
}
```

Response:

```
{
  "success": true,
  "userId": 123456789,
  "enabled": true
}
```

Get WanderGuide Status

Get current settings and user status.

```
GET /api/wanderguide/status?userId=123456789
```

Response (with userId):

```
{
  "userId": 123456789,
  "name": "eran",
  "enabled": true,
  "lat": 37.5796,
  "lng": 126.9770,
  "pushover": true,
  "radius": 300,
  "cooldown": 20,
```

```
  "revisit": 4
}
```

Response (without userId — global settings):

```
{
  "radius": 300,        // meters
  "cooldown": 20,       // minutes
  "revisit": 4,         // hours
  "users": 3            // total registered users
}
```

Update Global Settings

Change alert radius, cooldown, and revisit time (affects all users).

```
POST /api/wanderguide/settings
```

Request Body (all fields optional):

```
{
  "radius": 500,       // meters (50-5000)
  "cooldown": 10,      // minutes between any alerts (1-120)
  "revisit": 2         // hours before same POI re-alerts (0.5-24)
}
```

Response:

```
{
  "success": true,
  "radius": 500,
  "cooldown": 10,
  "revisit": 2
}
```

---

2. Alerts & Notifications

Poll for Proximity Alerts

Get new proximity alerts since a given timestamp.

```
GET /api/alerts?since=1713400000000
```

Parameters:

- `since` — Unix timestamp in milliseconds. Only returns alerts after this time.

Response:

```
[
  {
    "text": "📍 <b>Eran is 150m from Jogyesa Temple!</b> <span style=\"color:#888;font-size:12px\">2:30
PM</span><br>Chief Buddhist temple...<br><a href=\"https://www.google.com/maps/@37.573,126.9836,17z\">🗺️ Open
in Maps</a>",
```

```
    "time": 1713400123456
  }
]
```

Notes:

- Poll every 10-15 seconds
- Store the latest `time` value and pass it as `since` on next poll
- The `text` field contains HTML — strip tags for plain text display

---

Send Push Notification

To a specific user:

```
POST /api/push
```

Request Body:

```
{
  "to": "eran",                        // Registered user name
  "message": "Hey, meet at the hotel!"
}
```

Broadcast to all users:

```
POST /api/push
```

Request Body:

```
{
  "to": "all",
  "message": "Dinner at 7pm at Maple Tree House!"
}
```

Response:

```
{
  "success": true,
  "sentTo": "eran"                  // or ["eran", "michal", "jonathan", "ofir"]
}
```

Error (user not found):

```
{
  "error": "User \"bob\" not found. Registered: eran, michal, jonathan, ofir"
}
```

Register Pushover Key (Web API)

Register a Pushover user key for push notifications.

```
POST /api/pushover/register
```

Request Body:

```
{
  "name": "eran",
  "userKey": "uid2drphnhy...",
  "sendTest": true                  // Optional. Send a test notification
}
```

Response:

```
{
  "success": true,
  "registeredName": "eran",
  "users": ["eran", "michal"]
}
```

List Registered Pushover Users

```
GET /api/pushover/users
```

Response:

```
["eran", "michal", "jonathan", "ofir"]
```

---

3. POI Database

Get All POIs

Returns the complete POI database (static + custom).

```
GET /api/pois
```

Response:

```
[
  {
    "name": "Gyeongbokgung Palace",
    "lat": 37.5796,
    "lng": 126.9770,
    "desc": "Korea's grandest palace (1395). Guard changing ceremony at 10am & 2pm.",
    "city": "Seoul",
    "category": "palace"
  },
  ...
]
```

Notes:

- Returns 80+ static POIs + any custom POIs
- Categories: temple, palace, market, food, nature, culture, shopping, nightlife, viewpoint, hidden-gem

---

Add a Custom POI

```
POST /api/pois
```

Request Body:

```
{
  "name": "Amazing Tea House",
  "lat": 37.5745,
  "lng": 126.9856,
  "desc": "Hidden tea house on Insadong 5-gil. Try the citron tea."
}
```

Response:

```
{
  "success": true,
  "total": 85                        // Total POIs now in database
}
```

Notes:

- Custom POIs are persisted in PostgreSQL (survive server restarts)
- All users see custom POIs (shared database)
- Immediately available for proximity alerts

Clear All Custom POIs

Removes all custom POIs. Static POIs (80+) are unaffected.

```
DELETE /api/pois
```

Response:

```
{
  "success": true
}
```

---

4. Chat (AI Assistant)

Send a Chat Message

```
POST /api/chat
```

Headers:

```
Content-Type: application/json
X-Session-Id: unique_user_session_id
```

Request Body (text only):

```
{
  "message": "What's the plan for April 30?"
}
```

Request Body (text + image):

```
{
  "message": "Translate this sign",
  "image": {
    "data": "base64_encoded_image_data...",
    "mediaType": "image/jpeg"
  }
}
```

Response:

```
{
  "reply": "On April 30, you're heading to Gyeongju! Here's the plan:\n• 9:15 AM — Yonggungsa Temple..."
}
```

Notes:

- Each `X-Session-Id` gets its own conversation history (last 30 messages)
- The AI knows the full trip itinerary, voucher locations, and can answer questions about the trip
- Supports web search for current information
- Image support for translating Korean signs, reading menus, etc.
- Max image size: 10MB

---

5. Broadcasts (Scheduled Messages)

Poll for Broadcast Messages

Get scheduled messages (morning briefing, evening reminder, etc.)

```
GET /api/broadcasts?since=1713400000000
```

Parameters:

- `since` — Unix timestamp in milliseconds

Response:

```
[
  {
    "text": "🌅 Good morning! Here's today's plan...",
    "time": 1713400123456
  }
]
```

Notes:

- Broadcasts are generated automatically at 8am, 8pm, 10pm Korea time
- Poll every 30-60 seconds

---

6. Health Check

```
GET /api/health
```

Response:

```
{
  "status": "ok",
  "pois": 85,
  "sessions": 3
}
```

Typical Native App Flow

On First Launch

1. `POST /api/wanderguide/register` with userName + pushoverKey → store returned `userId`
2. Verify test push notification received

Ongoing (every 15-30 seconds while app is active)

3. Get device GPS coordinates
4. `POST /api/wanderguide/location` with userId + lat + lng
5. `GET /api/alerts?since=lastAlertTime` → display any new alerts

Periodic (every 30-60 seconds)

6. `GET /api/broadcasts?since=lastBroadcastTime` → display scheduled messages

User Actions

7. `POST /api/push` with `to` + `message` → send push to user or broadcast
8. `POST /api/pois` → add a custom POI
9. `POST /api/chat` → ask the AI assistant a question
10. `GET /api/pois` → refresh POI database (every 5 minutes)

Settings

11. `POST /api/wanderguide/toggle` → enable/disable alerts
12. `POST /api/wanderguide/settings` → change radius/cooldown/revisit
13. `GET /api/wanderguide/status` → check current settings

Data Persistence

Data	Storage	Survives Restart?
Static POIs (80+)	Code (poi-database.js)	✅ Always
Custom POIs	PostgreSQL	✅ Yes
User registrations	PostgreSQL	✅ Yes
Alert history	PostgreSQL	✅ Yes
Chat sessions	Server memory	❌ No
Alert queue	Server memory	❌ No
Broadcast queue	Server memory	❌ No

Error Handling

All endpoints return JSON. Errors use standard HTTP status codes:

Code	Meaning
200	Success
400	Bad request (missing required fields)
404	User not found (for push)
500	Server error

Error response format:

```
{
  "error": "Description of what went wrong"
}
```

Generated April 18, 2026 — Korea Trip Companion